

Facility: Adventure Bakery	Host Infrastructure: Hostinger VPS (srv1472620)
Database Backend: Supabase (PostgreSQL)	Service Architecture: Permanent Python MQTT Daemon

System Overview

This deployment establishes a permanent, secure data-collection pipeline mapping telemetry from physical YoLink wireless environment sensors directly into a central Supabase relational table. An asynchronous background process runs under systemd supervision on a dedicated Hostinger Virtual Private Server. The captured readings power the Team Portal **Temperature Monitoring** report (Compliance -> Temperature Logs).

Source Code & Repository

The logger source is version-controlled in the Adventure Bakery Team App repository. The copy running on the VPS should always match what is in git — treat git as the source of truth and deploy changes from there.

Item	Location
GitHub repository	github.com/Adv-bakes/Adventure-Bakery-Team-App
Path within repo	vps/yolink-logger/
Tracked files	main.py • requirements.txt • .env.example • .gitignore • README.md
Live copy on VPS	/root/yolink-logger/ (running under systemd)

Important: the real .env (YoLink UAID/Secret, Supabase service-role key) is **never** committed to git — it exists only on the VPS. The repo ships a .env.example listing the required variable names, and the repo README.md has full setup and deploy steps.

Deploy an updated main.py from git to the server

```
# Option A - copy the file straight up
scp vps/yolink-logger/main.py root@srv1472620:~/yolink-logger/main.py
ssh root@srv1472620 'sudo systemctl restart yolink-logger.service'

# Option B - if the VPS folder is itself a git clone
cd ~/yolink-logger && git pull && sudo systemctl restart yolink-logger.service
```

Hardware & Database Mapping

The Python application maps arriving telemetry packets from hex-encoded physical hardware IDs to human-readable names (via DEVICE_MAPPING in main.py) before validating parameters and inserting rows. Update that dictionary when a sensor is added or replaced.

Device Hardware ID	Mapped Room Label	Database Assignment
d88b4c010010b5da	Walk-In Refrigerator	public.temperature_logs
d88b4c010010b513	Walk-In Freezer	public.temperature_logs
d88b4c010010b70a	Bakery Floor	public.temperature_logs

Operational Architecture Deployment

1. Project Directory & Environment Structure

```
/root/yolink-logger/  
|-- venv/           isolated Python deps (paho-mqtt v2, supabase-py)  
|-- .env            protected credentials (NOT in git)  
|-- main.py         execution core script
```

2. Linux Background Daemon Integration (systemd)

Defined at /etc/systemd/system/yolink-logger.service. Restart=always revives the process after crashes or reboots.

```
[Unit]  
Description=YoLink MQTT Temperature Logger to Supabase  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
Type=simple  
User=root  
WorkingDirectory=/root/yolink-logger  
ExecStart=/root/yolink-logger/venv/bin/python main.py  
Restart=always  
RestartSec=10  
  
[Install]  
WantedBy=multi-user.target
```

■ CRITICAL RESET & RESTART CHEAT SHEET

Use these operational diagnostics if readings stall in Supabase, after a server reboot, or when credentials change. **Restarting the service fixes most problems**, because it forces a fresh YoLink token.

Linux System Action	Terminal Command to Execute
Restart the service (apply changes / fix lockups)	sudo systemctl restart yolink-logger.service
Check active stream status (is it green/running?)	sudo systemctl status yolink-logger.service
View live execution logs (debug packet arrivals)	sudo journalctl -u yolink-logger.service -f
Stop the daemon (perform code maintenance)	sudo systemctl stop yolink-logger.service
Reload background configs (if modifying service files)	sudo systemctl daemon-reload

Standard Troubleshooting Protocols

A. MQTT “Not authorized” / readings stop after ~2 hours

Symptom: the logs show Connected to YoLink MQTT Broker with result code Not authorized, and rows stop landing in temperature_logs — often after the service has been up for a couple of hours.

Cause: a YoLink access token is valid for only **2 hours (7200s)** and is used as the MQTT username. If the client reconnects using the *same* cached token after it has expired, the broker rejects it as “Not authorized” on every retry.

Fix (already in main.py): the daemon proactively fetches a **fresh token and rebuilds the connection every 100 minutes** (TOKEN_REFRESH_SECONDS), staying under the 2-hour expiry, and refuses to “subscribe” on a failed connection. If you see this again: first **restart the service** (forces a new token). If it recurs *immediately* after a restart — not hours later — the credentials are stale; re-issue them (item B) and update .env.

B. YoLink Credential (UAID / Secret) Problems

A token response of {"state": "error", "msg": "client_id not existed"} means the UAID sent was blank or unrecognized. Re-issue credentials in the YoLink app: **Settings -> Account -> Advanced Settings -> User Access Credentials** (re-issuing invalidates the previous pair), then update YOLINK_UAID and YOLINK_SECRET_KEY in .env and restart. Verify a token can be obtained (run on the VPS with the values loaded):

```
curl -s -X POST https://api.yosmart.com/open/yolink/token \
-d "grant_type=client_credentials&client_id=$YOLINK_UAID&client_secret=$YOLINK_SECRET_KEY"
```

A JSON body containing access_token = credentials good; an error body = env/credential problem. Note: .env is read by the *service*, not your login shell — echo \$YOLINK_UAID in a plain terminal looks empty even when the service is configured correctly.

C. Database Authentication Failures (401 Errors)

Verify that the secret **service_role** API key remains populated inside .env (SUPABASE_KEY). The anon key is blocked by Row-Level-Security and will fail every insert; it is never safe to expose.

D. Sensors Not Updating

YoLink equipment transmits selectively on temperature change or on a 30–60 min heartbeat to conserve battery, so gaps are normal. Press the physical **SET** button on the sensor to force an instant broadcast. If one sensor is persistently silent, check its battery gauge on the Temperature Monitoring report — a low battery (level 1) can stop transmissions.

E. Verifying Raw Transmission Packets

Stop the daemon and run the script by hand inside the virtualenv to watch raw JSON hit the pipeline live; restart the service when finished:

```
sudo systemctl stop yolink-logger.service
cd /root/yolink-logger && source venv/bin/activate && python main.py
# Ctrl-C when done, then:
sudo systemctl start yolink-logger.service
```

First-Response Quick Guide

If you see...	Do this first
Report data looks stale / no recent readings	systemctl status, then restart the service; watch journalctl -f

If you see...	Do this first
Logs say "Not authorized"	Restart service (new token). If instant on restart, re-issue YoLink credentials.
Logs say insert / 401 / RLS error	Check SUPABASE_KEY is the service-role key in .env
One sensor missing, others fine	Press SET on that sensor; check its battery level on the report
After a server reboot	Service auto-starts (Restart=always); confirm with systemctl status